Python Basics:

1. What is Python, and why is it popular?
- Answer: Python is a high-level, interpreted programming language known for its simplicity and readability. It is popular for web development, data analysis, machine learning, and more.

2. How do you comment in Python?
- Answer: You can use `#` for single-line comments and `""` or `"""` for multi-line comments.

3. What is the difference between Python 2 and Python 3?
- Answer: Python 3 is the latest version and has backward-incompatible changes from Python 2, with improved Unicode support and other enhancements.

4. Explain Python's main data types.
- Answer: Python's main data types include int, float, str, bool, list, tuple, set, and dict.

5. How do you declare and assign a variable in Python?
- Answer: You declare a variable by assigning a value to it, e.g., `x = 10`.

6. What is a tuple, and how is it different from a list?
- Answer: A tuple is an ordered, immutable sequence of elements, while a list is mutable.

7. Explain list comprehension in Python.
- Answer: List comprehension is a concise way to create lists by applying an expression to each item in an iterable.

8. How do you swap the values of two variables without using a temporary variable?
- Answer: You can use tuple packing and unpacking: `a, b = b, a`.

9. Describe Python's garbage collection.
- Answer: Python uses automatic garbage collection to manage memory by deallocating objects no longer in use.

10. What is the `None` value in Python?
- Answer: `None` represents the absence of a value or a null value in Python.

Control Flow:

11. What is an if statement in Python?
- Answer: An if statement is used for conditional execution. It runs a block of code if a condition is True.

12. What are "and" and "or" operators used for in Python?
- Answer: `and` returns True if both operands are True, while `or` returns True if at least one operand is True.

13. Explain the purpose of the `elif` statement in Python.
- Answer: `elif` is used to specify multiple conditions in an if-elif-else block, allowing alternative execution paths.

14. What is a "for" loop in Python?
- Answer: A `for` loop iterates over a sequence (e.g., a list) and executes a block of code for each item.

15. How do you terminate a loop prematurely in Python?
- Answer: You can use the `break` statement to exit a loop prematurely.

16. Explain the `range()` function in Python.
- Answer: `range()` generates a sequence of numbers, often used in for loops.

17. What is a "while" loop, and how is it different from a "for" loop?
- Answer: A `while` loop repeatedly executes a block of code while a condition is True, whereas a `for` loop iterates over a sequence.

18. How do you handle exceptions in Python?
- Answer: You can use a `try` and `except` block to catch and handle exceptions.

19. What is the purpose of the `finally` block in exception handling?
- Answer: The `finally` block is used to execute code regardless of whether an exception occurred or not.

20. Explain Python's `assert` statement.
- Answer: `assert` is used for debugging. It raises an error if a given condition is False.

Functions:

21. How do you define a function in Python?
- Answer: You define a function using the `def` keyword, e.g., `def my_function():`.

22. What is the difference between parameters and arguments in a function?
- Answer: Parameters are variables in a function's definition, while arguments are values passed when calling the function.

23. Explain the concept of a "return" statement in Python functions.
- Answer: A `return` statement is used to specify the value a function should return when it's called.

24. How can you define default parameter values in Python functions?
- Answer: You can define default values in the function's parameter list, e.g., `def greet(name="Guest"):`.

25. Describe variable scope in Python.

- Answer: Variables defined in a function have local scope, while those defined outside have global scope.

26. What is a lambda function in Python?
- Answer: A lambda function is a small, anonymous function defined using the `lambda` keyword.

27. Explain the concept of function closures in Python.
- Answer: A closure is a function that remembers values in the enclosing scope even if they are not present in memory.

28. How do you pass a variable number of arguments to a function in Python?
- Answer: You can use `*args` for positional arguments and `**kwargs` for keyword arguments.

29. Describe the purpose of the `map()` and `filter()` functions in Python.
- Answer: `map()` applies a function to each item in an iterable, while `filter()` filters items based on a condition.

30. What is recursion in Python, and when is it useful?
- Answer: Recursion is a technique where a function calls itself. It's useful for solving problems with repetitive structures.

Data Structures:

31. What is a list in Python, and how do you create one?
- Answer: A list is an ordered collection of items. You create one using square brackets, e.g., `my_list = [1, 2, 3]`.

32. Explain the difference between a shallow copy and a deep copy of a list.
- Answer: A shallow copy copies the list itself, while a deep copy copies both the list and its elements.

33. What is a dictionary in Python, and how do you create one?
- Answer: A dictionary is an unordered collection of key-value pairs. You create one using curly braces, e.g., `my_dict = {"key": "value"}`.

34. How do you access and modify dictionary elements in Python?
- Answer: You can access elements using keys and modify them by assigning new values to keys.

35. Describe the purpose of a set in Python.
- Answer: A set is an unordered collection of unique elements used for various mathematical operations.

36. Explain the difference between a set and a frozenset.
- Answer: A set is mutable, while a frozenset is immutable and can be used as a key in

dictionaries.

37. What is a tuple in Python, and how do you create one?
- Answer: A tuple is an ordered, immutable collection of elements created using parentheses, e.g., `my_tuple = (1, 2, 3)`.

38. How do you add and remove items from a list in Python?
- Answer: You can use the `append()` method to add items and the `remove()` method to remove items from a list.

39. Explain the purpose

of list slicing in Python.
- Answer: List slicing allows you to create a new list by extracting a portion of an existing list.

40. What are list comprehensions, and how do you use them?
- Answer: List comprehensions provide a concise way to create lists by applying an expression to each item in an iterable.

Object-Oriented Programming (OOP):

41. What is object-oriented programming (OOP)?
- Answer: OOP is a programming paradigm that uses objects (instances of classes) to model real-world entities.

42. What is a class in Python, and how do you define one?
- Answer: A class is a blueprint for creating objects. You define one using the `class` keyword.

43. Explain the difference between a class and an object.
- Answer: A class is a template for objects, while an object is an instance of a class.

44. How do you create an instance (object) of a class in Python?
- Answer: You create an object by calling the class as if it were a function, e.g., `my_obj = MyClass()`.

45. Describe encapsulation in Python's OOP.
- Answer: Encapsulation is the practice of bundling data (attributes) and methods (functions) that operate on the data into a single unit (class).

46. What is inheritance in OOP, and how does it work in Python?
- Answer: Inheritance allows a class to inherit attributes and methods from another class. It promotes code reuse and establishes a parent-child relationship.

47. Explain polymorphism in OOP, and provide an example in Python.
- Answer: Polymorphism allows objects of different classes to be treated as objects of the same class. For example, you can use different classes with a common interface interchangeably.

48. What is method overloading in Python, and how is it achieved?
- Answer: Python doesn't support method overloading like some other languages. Instead, you can use default argument values to achieve similar functionality.

49. Describe the concept of method overriding in OOP.
- Answer: Method overriding allows a subclass to provide a specific implementation of a method that is already defined in its parent class.

50. What is a constructor in Python, and how is it defined?
- Answer: A constructor is a special method that gets called when an object is created. In Python, the constructor is defined as `__init__()`.

Modules and Packages:

51. What is a module in Python?
- Answer: A module is a file containing Python code. It can define functions, classes, and variables.

52. How do you import a module in Python?
- Answer: You can import a module using the `import` statement, e.g., `import math`.

53. Explain the purpose of the `if __name__ == "__main__":` construct.
- Answer: This construct is used to check if a Python script is being run as the main program or if it's being imported as a module.

54. What is a package in Python, and how is it different from a module?
- Answer: A package is a collection of related modules organized in directories. It provides a way to structure and manage larger Python projects.

55. How do you create and use your own packages in Python?
- Answer: You create packages by organizing modules in directories with a special `__init__.py` file. You can then import and use them like any other module.

File Handling:

56. How do you open and close files in Python?
- Answer: You can open a file using the `open()` function and close it using the `close()` method or the `with` statement.

57. Explain the purpose of the "r", "w", and "a" modes when opening files.
- Answer: "r" is for reading, "w" is for writing (creates a new file or overwrites an existing one), and "a" is for appending to an existing file.

58. How do you read the contents of a file in Python?
- Answer: You can use methods like `read()`, `readline()`, or `readlines()` to read the contents of

a file.

59. What is the purpose of the "with" statement in file handling?
- Answer: The `with` statement is used to ensure that a file is properly closed after its suite finishes executing.

60. How do you write data to a file in Python?
- Answer: You can use methods like `write()` to write data to a file in text mode or `wb` in binary mode.

Exception Handling:

61. What is an exception in Python?
- Answer: An exception is an event that occurs during the execution of a program, disrupting the normal flow of instructions.

62. How do you handle exceptions in Python?
- Answer: You use a `try` and `except` block to catch and handle exceptions.

63. What is the purpose of the `finally` block in exception handling?
- Answer: The `finally` block is used to execute code regardless of whether an exception occurred or not.

64. How can you raise a custom exception in Python?
- Answer: You can raise a custom exception using the `raise` statement, e.g., `raise ValueError("Custom error message")`.

65. Describe common built-in exceptions in Python.
- Answer: Common exceptions include `ZeroDivisionError`, `ValueError`, `TypeError`, `FileNotFoundError`, and `KeyError`.

Regular Expressions:

66. What are regular expressions (regex) in Python?
- Answer: Regular expressions are patterns used for searching and matching text.

67. How do you use the `re` module for regular expressions in Python?
- Answer: You import the `re` module and use its functions like `search()`, `match()`, and `findall()` to work with regular expressions.

68. Explain the purpose of character classes in regular expressions.
- Answer: Character classes allow you to match sets of characters, e.g., `[0-9]` matches any digit.

69. What is the difference between `match()` and `search()` functions in the `re` module?
- Answer: `match()` matches the pattern only at the beginning of the string, while `search()` searches for the pattern anywhere in the string.

70. How do you use capturing groups in regular expressions?
- Answer: You use parentheses to create capturing groups that allow you to extract specific parts of a matched string.

File I/O and Data Serialization:

71. What is JSON, and how do you work with it in Python?
- Answer: JSON (JavaScript Object Notation) is a lightweight data interchange format. Python has a `json` module for encoding and decoding JSON data.

72. How do you read and write JSON files in Python?
- Answer: You can use `json.load()` to read JSON from a file and `json.dump()` to write JSON to a file.

73. Explain the purpose of the `pickle` module in Python.
- Answer: The `pickle` module is used for serializing and deserializing Python objects, allowing them to be saved to and loaded from files.

74. How do you open and close files in binary mode for non-textual data?
- Answer: You open files in binary

mode by adding "b" to the file mode, e.g., "rb" for reading binary and "wb" for writing binary.

75. What is the difference between JSON and pickle for data serialization?
- Answer: JSON is a text-based format used for data interchange, while pickle is a binary format for serializing Python objects.

Modules and Libraries:

76. How do you install external Python libraries using pip?
- Answer: You use the `pip` tool to install packages, e.g., `pip install package-name`.

77. What is a virtual environment in Python?
- Answer: A virtual environment is a self-contained directory that contains a Python interpreter and a set of libraries.

78. How do you create and activate a virtual environment in Python?
- Answer: You use the `venv` module to create a virtual environment, and then activate it using the `source` command or `activate` script.

79. Explain the purpose of the `sys` module in Python.
- Answer: The `sys` module provides functions and variables to interact with the Python runtime environment.

80. How do you import and use modules in Python?

- Answer: You use the `import` statement to import modules and access their functions, classes, and variables.

Concurrent Programming:

81. What is the Global Interpreter Lock (GIL) in Python, and how does it impact multi-threading?
- Answer: The GIL is a mutex that allows only one thread to execute in the interpreter at a time, limiting multi-threading performance.

82. How do you create and manage threads in Python using the `threading` module?
- Answer: You can create and manage threads using the `threading` module by defining a thread class and starting threads.

83. What is the difference between multi-threading and multi-processing in Python?
- Answer: Multi-threading uses multiple threads in a single process, while multi-processing creates multiple processes, each with its own Python interpreter.

84. How do you use locks (mutexes) to synchronize threads in Python?
- Answer: You can use the `Lock` class from the `threading` module to create locks and ensure safe access to shared resources.

85. Explain the purpose of the `queue` module in Python's multi-threading.
- Answer: The `queue` module provides thread-safe data structures like `Queue` for communication between threads.

Decorators and Generators:

86. What is a decorator in Python?
- Answer: A decorator is a function that wraps another function to extend or modify its behavior.

87. How do you define and use decorators in Python?
- Answer: You define decorators by using the `@` symbol above a function and apply them to other functions.

88. What are generator functions in Python?
- Answer: Generator functions are functions that use the `yield` keyword to yield values one at a time, allowing for efficient memory usage.

89. How do you create a generator object and iterate over its values?
- Answer: You create a generator object by calling a generator function. You can iterate over its values using a `for` loop or by calling `next()`.

90. Explain the difference between a generator and a regular function.
- Answer: A regular function runs to completion and returns a single value, while a generator function can yield multiple values and retains its state between calls.

Python Libraries:

91. What is NumPy, and why is it important in data science?
- Answer: NumPy is a Python library for numerical computing. It provides arrays and mathematical functions crucial for data science and scientific computing.

92. Describe the purpose of the Pandas library in Python.
- Answer: Pandas is a library for data manipulation and analysis. It provides data structures like DataFrames and Series for handling tabular data.

93. How do you install and use Matplotlib for data visualization in Python?
- Answer: You install Matplotlib using `pip` and use it to create various types of plots, charts, and graphs.

94. What is the purpose of the scikit-learn library in Python?
- Answer: Scikit-learn is a machine learning library used for tasks like classification, regression, clustering, and model evaluation.

95. Explain the role of TensorFlow and PyTorch in Python's machine learning ecosystem.
- Answer: TensorFlow and PyTorch are deep learning frameworks used for developing and training neural networks.

Web Development and APIs:

96. How do you make HTTP requests in Python?
- Answer: You can use libraries like `requests` to make HTTP requests and interact with APIs.

97. What is Flask, and how do you create a web application with it?
- Answer: Flask is a micro web framework for Python. You create a Flask web application by defining routes and views in Python.

98. Describe the purpose of Django in Python web development.
- Answer: Django is a high-level web framework that provides a complete toolkit for building web applications, including an ORM and authentication.

99. How do you work with JSON data in Python when interacting with web APIs?
- Answer: You can use the `json` module to parse JSON responses from web APIs into Python objects and serialize Python objects to JSON.

100. What is a RESTful API, and how do you consume one in Python?
- Answer: A RESTful API is an architectural style for designing networked applications. You can consume one in Python using HTTP requests and libraries like `requests`.

These 100 Python interview questions cover a wide range of topics and should help you assess a candidate's knowledge and proficiency in Python programming. Please note that candidates

should not only provide answers but also be prepared to discuss their thought processes and problem-solving skills during the interview.